



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»

Школа разработки видеоигр

Практическое занятие №3

«Менеджер ресурсов и загрузка графических ассетов»
по дисциплине «Архитектура игровых движков»

Выполнили:

студенты группы J3300

Власов Д. Н.

Шабанов Т. О.

Проверил:

доцент практики

Павлович А. А.

г. Санкт-Петербург

2026 год

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1. Цель работы.....	3
2. Задачи работы	3
ХОД РАБОТЫ.....	4
1. Описание и архитектура.....	4
2. Технологический стек.....	4
3. Система сборки	4
4. Менеджер ресурсов.....	5
5. Загрузчик мешей.....	5
6. Загрузчик текстур	6
7. Загрузчик шейдеров	7
8. Расширение RenderAdapter.....	7
9. Создание GPU-ресурсов	8
10. Модификация компонента MeshRenderer	8
11. Адаптация системы рендеринга	9
12. Сериализация сцены.....	9
13. Демонстрационная сцена и загрузка модели	10
14. Трудности и их решение.....	11
ЗАКЛЮЧЕНИЕ.....	12
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	13
ПРИЛОЖЕНИЕ 1	
ПРИЛОЖЕНИЕ 2	
ПРИЛОЖЕНИЕ 3	

ВВЕДЕНИЕ

1. Цель работы

Целью данного практического занятия является разработка централизованной системы управления ресурсами (моделями, текстурами, шейдерами) с кэшированием, интегрирование загрузки 3D-моделей в игровой движок и обеспечение их визуализации через компонент MeshRenderer на базе ECS.

2. Задачи работы

Задачами данного практического занятия являются:

- 1) Подключение и настройка библиотеки для загрузки моделей (Assimp / tinyobjloader), изображений (stb_image) и работы с JSON (nlohmann/json).
- 2) Реализация шаблонного класса ResourceManager с кэшированием загруженных ресурсов.
- 3) Создание загрузчиков для конкретных типов ресурсов: Mesh, Texture, Shader.
- 4) Расширение RenderAdapter методами создания GPU-буферов, текстур и шейдерных программ.
- 5) Модифицирование компонента MeshRenderer для использования идентификаторов/ссылок на загруженные ресурсы
- 6) Адаптирование RenderSystem для отрисовки объектов с загруженными мешами и текстурами.
- 7) Демонстрация загрузки и отображения нескольких 3D-моделей с текстурами.
- 8) Реализация горячей замены шейдеров/конфигураций по изменению файла.

ХОД РАБОТЫ

1. Описание и архитектура

В ходе практического занятия архитектура игрового движка NeneEngine, созданная в предыдущих работах, была расширена полноценной подсистемой управления ресурсами. Введён центральный ResourceManager, который хранит кэш ресурсов, сопоставляя тип ресурса и путь к файлу с уже загруженным объектом.

Новая подсистема интегрирована с существующими ECS и рендерингом. Загрузчики подготавливают CPU-представление ресурса, после чего RenderAdapter создаёт соответствующие GPU-объекты. В MeshRendererComponent вместо упрощённого описания примитива используются идентификаторы меша и материала, а RenderSystem передаёт эти данные в адаптер рендеринга через структуру renderItem.

Ссылка на репозиторий с исходным кодом игрового движка NeneEngine на платформе хостинга репозитория GitHub представлена в [Приложении 1](#).

2. Технологический стек

Для выполнения практического занятия технологический стек проекта был расширен следующими средствами:

- 1) Assimp для импорта 3D-моделей из файлов формата .obj.
- 2) stb_image для проверки загрузки текстур и работы с изображениями.

3. Система сборки

Подключение зависимостей выполнено через vcpkg.json и src/CMakeLists.txt. В CMake-конфигурации используются find_package(assimp CONFIG REQUIRED), find_package(nlohmann_json CONFIG REQUIRED) и find_package(Stb MODULE REQUIRED), а также линковка с Diligent-TextureLoader.

Для проверки корректности интеграции внешних библиотек реализован модуль ExternalLibrarySmokeTest, который при запуске приложения выполняет

тестовую загрузку OBJ-модели через ResourceManager и текстуры через stb_image, после чего выводит результаты в лог.

Полный листинг корневого CMake-файла CMakeLists.txt продемонстрирован в [Приложении 2](#).

Полный листинг CMake-файла CMakeLists.txt в директории src продемонстрирован в [Приложении 3](#).

4. Менеджер ресурсов

Центральным компонентом новой подсистемы стал шаблонный ResourceManager. Он хранит набор зарегистрированных загрузчиков по типам ресурсов, а также кэш уже загруженных объектов. При запросе ресурса по пути менеджер сначала проверяет наличие объекта в кэше; если ресурс уже был загружен, возвращается готовый shared_ptr, иначе вызывается соответствующий загрузчик и результат сохраняется.

Проверка кэша и загрузка ресурсов приведены в листинге 1. Регистрация стандартных загрузчиков приведена в листинге 2.

Листинг 1 – проверка кэша и загрузка ресурсов

```
template <typename T> ResourcePtr<T> Load(const std::string& path)
{
    auto& cache = GetOrCreateCache<T>();
    if (const auto cached = cache.find(path); cached != cache.end())
        return cached->second;

    LoaderFn<T>* loader = GetLoader<T>();
    auto resource = std::make_shared<Resource<T>>(path, (*loader)(path));
    cache.emplace(path, resource);
    return resource;
}
```

Листинг 2 – регистрация стандартных загрузчиков

```
void ResourceManager::RegisterDefaultLoaders()
{
    RegisterLoader<Mesh>([](const std::string& path) { return LoadMeshFromFile(path);
});
    RegisterLoader<TextureResource>([](const std::string& path) { return
LoadTextureResourceFromFile(path); });
    RegisterLoader<ShaderProgramResource>(
        [](const std::string& path) { return LoadShaderProgramResourceFromFile(path);
});
}
```

5. Загрузчик мешей

Для загрузки 3D-моделей был реализован модуль MeshLoader,

использующий библиотеку Assimp. При чтении модели применяются этапы постобработки: триангуляция, объединение одинаковых вершин, генерация нормалей, проверка структуры данных и инвертирование UV-координат.

Из сцены Assimp последовательно извлекаются вершины и индексы, после чего формируется структура MeshData. На её основе создаётся объект Mesh, который в дальнейшем может быть выгружен на GPU.

Проверка чтения модели через Assimp приведена в листинге 3. Заполнение вершин приведено в листинге 4.

Листинг 3 – чтение моделей через Assimp

```
const aiScene* scene = importer.ReadFile(
    path, aiProcess_Triangulate | aiProcess_JoinIdenticalVertices |
        aiProcess_PreTransformVertices | aiProcess_GenSmoothNormals |
        aiProcess_ImproveCacheLocality | aiProcess_ValidateDataStructure |
        aiProcess_FlipUVs);
```

Листинг 4 – заполнение вершин

```
Vertex vertex{};
vertex.position = {position.x, position.y, position.z};
vertex.normal = {normal.x, normal.y, normal.z};
vertex.uv = {uv.x, uv.y};
meshData.vertices.push_back(vertex);
```

6. Загрузчик текстур

Для работы с текстурами введена структура TextureResource и модуль TextureLoader. Текстура представлена путём к файлу, флагом sRGB и режимом фильтрации. Параметры фильтрации считываются из дополнительного JSON-файла texture.json, который хранится рядом с изображением.

Чтение метаданных структур приведено в листинге 5. Создание ресурса текстуры приведено в листинге 6.

Листинг 5 – чтение метаданных структур

```
const nlohmann::json metadata = nlohmann::json::parse(file);
const std::string filter = metadata.value("filter", "linear");
return filter == "nearest" || filter == "point"
    ? TextureFilterMode::Nearest
    : TextureFilterMode::Linear;
```

Листинг 6 – создание ресурса текстуры

```
const std::filesystem::path metadataPath =
    texturePath.parent_path() / (texturePath.stem().string() + ".texture.json");

return TextureResource{path, true, ReadFilterMode(metadataPath)};
```

7. Загрузчик шейдеров

Для шейдеров реализован manifest-подход: отдельный shader-файл в формате JSON хранит пути к вершинному и пиксельному шейдерам. Загрузчик ShaderLoader сначала читает JSON-описание программы, затем загружает исходный код обоих шейдеров в память.

Чтение описания шейдера приведено в листинге 7. Формирование ShaderProgramResource приведено в листинге 8.

Листинг 7 – чтение описания шейдера

```
const nlohmann::json description = nlohmann::json::parse(ReadTextFile(programPath));
const std::filesystem::path vertexPath =
    ResolveRelativeTo(programPath, description.at("vertex").get<std::string>());
const std::filesystem::path pixelPath =
    ResolveRelativeTo(programPath, description.at("pixel").get<std::string>());
```

Листинг 8 – формирование ShaderProgramResource

```
ShaderProgramResource resource{};
resource.vertexPath = vertexPath.string();
resource.pixelPath = pixelPath.string();
resource.vertexSource = ReadTextFile(vertexPath);
resource.pixelSource = ReadTextFile(pixelPath);
```

8. Расширение RenderAdapter

Интерфейс IRenderAdapter был расширен новыми методами, необходимыми для работы с полноценными графическими ресурсами: созданием вершинных и индексных буферов, загрузкой меша на GPU, созданием текстуры и компиляцией шейдерной программы.

Новые методы интерфейса приведено в листинге 9. Загрузка меша в DiligentDX12Adapter приведено в листинге 10.

Листинг 9 – новые методы интерфейса

```
virtual GPUBuffer CreateVertexBuffer(const void* vertexData, uint64_t sizeBytes,
uint32_t vertexCount) = 0;
virtual GPUBuffer CreateIndexBuffer(const uint32_t* indices, uint32_t indexCount) =
0;
virtual GPUMesh UploadMesh(const MeshData& meshData) = 0;
virtual GPUTexture CreateTexture2D(const TextureResource& texture) = 0;
virtual GPUShaderProgram CreateShaderProgram(const ShaderProgramResource&
shaderProgram) = 0;
```

Листинг 10 – загрузка меша в DiligentDX12Adapter

```
const GPUBuffer vertexBuffer =
    CreateVertexBuffer(meshData.vertices.data(), meshData.vertices.size() *
sizeof(Vertex),
static_cast<uint32_t>(meshData.vertices.size()));
```

```
const GPUBuffer indexBuffer =
    CreateIndexBuffer(meshData.indices.data(),
        static_cast<uint32_t>(meshData.indices.size()));
```

9. Создание GPU-ресурсов

В DiligentDX12Adapter были реализованы механизмы превращения CPU-представлений ресурсов в GPU-объекты. Для текстур используется Diligent TextureLoader, а для шейдеров создаются pipeline state object и связанные с ним ресурсы.

Создание текстуры приведено в листинге 11. Создание шейдерной программы приведено в листинге 12.

Листинг 11 – создание текстуры

```
RefCntAutoPtr<ITextureLoader> textureLoader;
CreateTextureLoaderFromFile(texture.path.c_str(), IMAGE_FILE_FORMAT_UNKNOWN,
    loadInfo, &textureLoader);
textureLoader->CreateTexture(m_pDevice, &uploadedTexture.texture);
```

Листинг 12 – создание шейдерной программы

```
shaderCI.Desc.ShaderType = SHADER_TYPE_VERTEX;
shaderCI.Source = shaderProgram.vertexSource.c_str();
m_pDevice->CreateShader(shaderCI, &vertexShader);

shaderCI.Desc.ShaderType = SHADER_TYPE_PIXEL;
shaderCI.Source = shaderProgram.pixelSource.c_str();
m_pDevice->CreateShader(shaderCI, &pixelShader);
```

10. Модификация компонента MeshRenderer

Компонент MeshRendererComponent был переработан под новую ресурсную модель. Теперь он хранит MeshId и структуру Material, внутри которой содержатся MaterialId, ShaderId, TextureId и цветовой множитель tint.

Структура компонента приведена в листинге 13. Структура материала приведена в листинге 14.

Листинг 13 – структура компонента

```
struct MeshRendererComponent
{
    PrimitiveType primitiveType = PrimitiveType::Triangle;
    bool visible = true;
    MeshId meshId{};
    Material material{};
};
```

Листинг 14 – структура материала

```
struct Material
{
    MaterialId materialId{};
```



```

ShaderId shaderId{};
TextureId textureId{};
glm::vec4 tint = {1.0f, 1.0f, 1.0f, 1.0f};
};

```

11. Адаптация системы рендеринга

Система `RenderSystem` была изменена так, чтобы для каждой сущности с `TransformComponent` и `MeshRendererComponent` формировать объект `RenderItem`, содержащий матрицы трансформации и ссылки на графические ресурсы. Затем этот объект передаётся в рендер-адаптер.

Заполнение `RenderItem` приведено в листинге 15.

Листинг 15 – заполнение `RenderItem`

```

RenderItem item{};
item.modelMatrix = ComputeWorldMatrix(world, entity, worldMatrixCache,
recursionStack);
item.modelViewProjectionMatrix = viewProjectionMatrix * item.modelMatrix;
item.meshId = meshRenderer.meshId;
item.shaderId = meshRenderer.material.shaderId;
item.textureId = meshRenderer.material.textureId;
item.tint = meshRenderer.material.tint;

```

12. Сериализация сцены

Дополнительно в проекте реализован `SceneSerializer`, сохраняющий и восстанавливающий ECS-сцену в формате JSON. Сериализуются трансформации, камера, иерархия, движение и параметры `MeshRenderer`, включая идентификаторы меша, шейдера и текстуры.

Сериализация `MeshRenderer` приведено в листинге 16. Сохранение сцены в файл приведено в листинге 17

Листинг 16 – сериализация `MeshRenderer`

```

return {{"primitiveType", ToString(renderer.primitiveType)},
{"visible", renderer.visible},
{"meshId", renderer.meshId.value},
{"material",
{"materialId", renderer.material.materialId.value},
{"shaderId", renderer.material.shaderId.value},
{"textureId", renderer.material.textureId.value},
{"tint", toJson(renderer.material.tint)}}};

```

Листинг 17 – сохранение сцены в файл

```

void SceneSerializer::SaveToFile(const ECS::World& world, const
std::filesystem::path& path)
{
    std::ofstream file(path);
    file << Serialize(world).dump(4);
}

```

13. Демонстрационная сцена и загрузка модели

В приложении реализована демонстрационная загрузка OBJ-модели momosuzu_nene_posed.obj. После старта движка модель загружается через ResourceManager, затем выгружается на GPU, для неё создаётся шейдерная программа и подбирается текстура из mtl-описания материала.

После этого в ECS создаётся отдельная сущность LoadedObjModel, которой назначаются трансформация и настроенный MeshRendererComponent.

Загрузка меша приведена в листинге 18. Создание сущности модели приведено в листинге 19.

Листинг 18 – загрузка меша

```
if (auto meshResource = ResourceManager::GetInstance().Load<Mesh>(meshPath.string());
    meshResource != nullptr)
{
    Mesh& mesh = meshResource->GetData();
    const GPUMesh gpuMesh = m_windows.front().renderer->UploadMesh(mesh.data);
}
```

Листинг 19 – создание сущности модели

```
const ECS::Entity modelEntity = m_world.CreateEntity("LoadedObjModel");
auto& modelRenderer = m_world.AddComponent<ECS::MeshRendererComponent>(modelEntity);
modelRenderer.meshId = gpuMesh.meshId;
modelRenderer.material.textureId = textureId;
modelRenderer.material.shaderId = shaderId;
```

Демонстрация работы игрового движка представлена на рисунке 1.

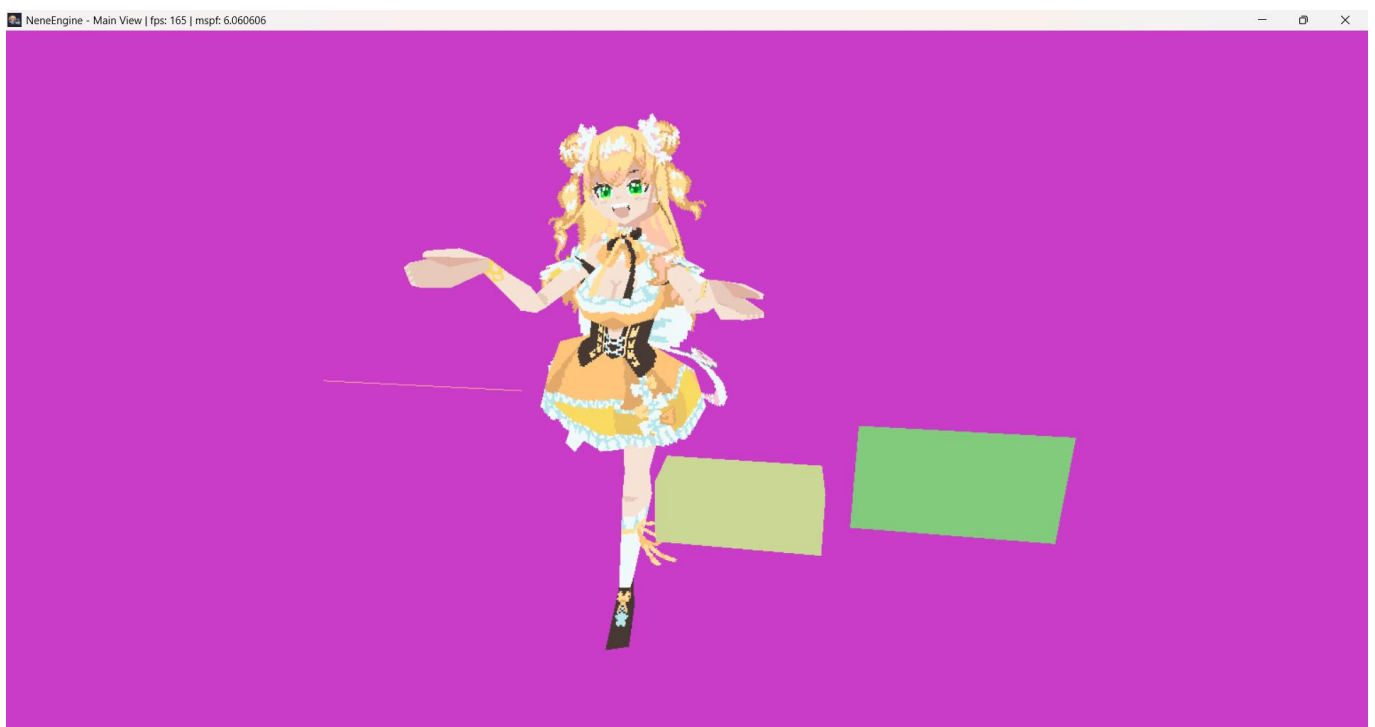


Рис. 1 Загрузка и отрисовка модели в демонстрационном окне

14. Трудности и их решение

Наиболее сложной частью работы стало согласование нескольких уровней представления ресурсов: файловых ассетов, CPU-структур, GPU-объектов и компонентов ECS. Для решения этой задачи была выбрана поэтапная схема: сначала ресурс загружается и кэшируется, затем создаётся его GPU-представление, после чего сущности получают только необходимые идентификаторы.

Отдельную трудность представляла интеграция старой схемы примитивов с новой системой мешей и материалов. Эта проблема была решена путём сохранения `PrimitiveType` для совместимости и одновременного добавления `MeshId` и `Material` в компонент рендеринга.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практического занятия №3 в игровом движке NeneEngine была разработана и интегрирована централизованная подсистема управления графическими ресурсами. В рамках работы были подключены и настроены библиотеки Assimp и stb_image, реализован шаблонный ResourceManager с кэшированием, созданы загрузчики мешей, текстур и шейдерных программ, а также расширен RenderAdapter методами создания GPU-ресурсов.

Кроме того, была модифицирована ECS-архитектура движка: компонент MeshRenderer переведён на использование идентификаторов мешей, текстур и шейдеров, а RenderSystem адаптирован для отрисовки объектов с загруженными ресурсами. В результате движок перешёл от отображения простых геометрических примитивов к поддержке полноценной загрузки и визуализации 3D-моделей с текстурами.

Работа выполнена в полном объёме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Джейсон Г. Игровой движок. – СПб.: Издательский дом Питер, 2021 г. – 1136 с.
2. Нильстрём Р. Паттерны программирования игр. – М.: Эксмо, 2021 г. – 432 с.
3. Купер Дж. Основы архитектуры игровых движков. – М.: Наука, 2018 г. – 2022 с.
4. <https://github.com/electronicarts/EASTL/tree/master/doc> – документация альтернативной стандартной библиотеки шаблонов EASTL.
5. <https://spdlog.readthedocs.io/en/latest/> – документация библиотеки логирования spdlog.
6. <https://diligentgraphics.github.io/docs/> – документация графического фреймворка Diligent Graphics.
7. <https://learn.microsoft.com/en-us/windows/win32/api/> – документация API оконной системы Windows Win32API.
8. <https://nlohmann.github.io/json/> – документация библиотеки для работы с JSON-форматом nlohmann/json.
9. <https://skypjack.github.io/entt/> – документация библиотеки Entity Component System EnTT.
10. <https://github.com/g-truc/glm/tree/master/doc> – документация математической библиотеки glm.
11. <https://the-asset-importer-lib-documentation.readthedocs.io/en/latest/> – документация библиотеки для загрузки ассетов assimp.
12. <https://github.com/nothings/stb/tree/master/docs> – документация библиотеки чтения изображений Stb.

Репозиторий игрового движка NeneEngine на платформе GitHub

<https://github.com/BennySpace/NeneEngine>

Листинг корневого CMake-файла CMakeLists.txt

```

# NeneEngine/CMakeList.txt : Top-level CMake project file, do global configuration

cmake_minimum_required(VERSION 3.19)

project(
    NeneEngine
    VERSION 0.2
    DESCRIPTION "Game engine"
    LANGUAGES CXX C
)

set(CMAKE_CXX_STANDARD          20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS       OFF)

# Diligent Engine configs
set(DILIGENT_BUILD_SAMPLES OFF CACHE BOOL "" FORCE)
set(DILIGENT_BUILD_TOOLS   OFF CACHE BOOL "" FORCE)
set(DILIGENT_BUILD_FX      OFF CACHE BOOL "" FORCE)
set(DILIGENT_BUILD_TESTS   OFF CACHE BOOL "" FORCE)
set(DILIGENT_INSTALL_TOOLS OFF CACHE BOOL "" FORCE)

set(DILIGENT_NO_DIRECT3D11 ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_VULKAN     ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_OPENGL     ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_GLES       ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_METAL      ON  CACHE BOOL "" FORCE)
set(DILIGENT_NO_DIRECT3D12 OFF CACHE BOOL "" FORCE)

add_subdirectory("external/DiligentEngine/DiligentCore")
set(DILIGENT_TOOLS_DIR "external/DiligentEngine/DiligentTools")
add_subdirectory("external/DiligentEngine/DiligentTools/ThirdParty")
add_subdirectory("external/DiligentEngine/DiligentTools/TextureLoader")

foreach(target_name SPIRV-Tools-opt SPIRV-Tools SPIRV-Tools-core)
    if(TARGET ${target_name})
        get_target_property(aliased_target ${target_name} ALIASED_TARGET)
        if(aliased_target)
            target_compile_options(${aliased_target} PRIVATE /wd5232 /wd4717)
            set_target_properties(${aliased_target} PROPERTIES
                COMPILE_WARNING_AS_ERROR OFF)
            message(STATUS "[Diligent Fix] Applied to aliased target:
${aliased_target}")
        else()
            target_compile_options(${target_name} PRIVATE /wd5232 /wd4717)
            set_target_properties(${target_name} PROPERTIES COMPILE_WARNING_AS_ERROR
                OFF)
            message(STATUS "[Diligent Fix] Applied to target: ${target_name}")
        endif()
    endif()
endforeach()

add_subdirectory("src")

set_property(DIRECTORY ${CMAKE_SOURCE_DIR} PROPERTY VS_STARTUP_PROJECT NeneEngine)

```

Листинг CMake-файла CMakeLists.txt в директории src/

```

# NeneEngine/src/CMakeList.txt : CMake project for NeneEngine, include source and
define project specific logic here.

add_executable(NeneEngine WIN32
    "${CMAKE_CURRENT_SOURCE_DIR}/../WinMain.cpp"
    "${CMAKE_CURRENT_SOURCE_DIR}/../NeneEngine.rc"
)

target_sources(NeneEngine PRIVATE
    "${CMAKE_CURRENT_SOURCE_DIR}/../include/Resource.h"
)

add_subdirectory(App)
add_subdirectory(Core)
add_subdirectory(ECS)
add_subdirectory(Input)
add_subdirectory(Platform)
add_subdirectory(RenderAdapters)
add_subdirectory(Rendering)
add_subdirectory(Scene)
add_subdirectory(States)

target_include_directories(NeneEngine
    PRIVATE
        $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/../include>
        $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/../external>
)

find_package(spdlog          CONFIG REQUIRED)
find_package(assimp          CONFIG REQUIRED)
find_package(nlohmann_json  CONFIG REQUIRED)
find_package(EASTL          CONFIG REQUIRED)
find_package(EnTT           CONFIG REQUIRED)
find_package(glm            CONFIG REQUIRED)
find_package(Stb            MODULE REQUIRED)

target_include_directories(NeneEngine PRIVATE ${Stb_INCLUDE_DIR})

target_link_libraries(NeneEngine
    PRIVATE
        assimp::assimp
        spdlog::spdlog
        nlohmann_json::nlohmann_json
        EASTL
        EnTT::EnTT
        glm::glm
        Diligent-Win32Platform
        Diligent-GraphicsEngineD3D12-shared
        Diligent-TextureLoader
)

if(EXISTS "${CMAKE_BINARY_DIR}/vcpkg_installed/x64-windows/debug/bin")
    file(GLOB NENEENGINE_VCPKG_DEBUG_DLLS "${CMAKE_BINARY_DIR}/vcpkg_installed/x64-
windows/debug/bin/*.dll")
    if(NENEENGINE_VCPKG_DEBUG_DLLS)
        add_custom_command(TARGET NeneEngine POST_BUILD
            COMMAND ${CMAKE_COMMAND} -E copy_if_different
                ${NENEENGINE_VCPKG_DEBUG_DLLS}
                $<TARGET_FILE_DIR:NeneEngine>
        )
    endif()
endif()

```



```
        COMMAND_EXPAND_LISTS
    )
endif()
endif()

if(COMMAND copy_required_dlls)
    copy_required_dlls(NeneEngine)
else()
    message(WARNING "copy_required_dlls was not found; Diligent may not be configured
correctly")
endif()

target_compile_features(NeneEngine PRIVATE cxx_std_20)
```